Approximate and Weighted Data Reconstruction Attack in Federated Learning

Yongcun Song, Ziqi Wang, and Enrique Zuazua

Abstract—Federated learning (FL) is a distributed learning paradigm that enables multiple clients to collaboratively train a machine learning model without sharing private data. Although FL is often regarded as privacy-preserving by design, recent data reconstruction attacks demonstrate that adversaries can recover clients' training data from shared model updates. However, existing attack methods frequently fail in the widely used federated averaging (FedAvg) setting, where clients transmit model parameters only after executing multiple local training steps. To overcome this limitation, we propose an interpolation-based approximation method that makes attacks on FedAvg feasible by effectively estimating the intermediate model updates generated during local training. Furthermore, we design a layer-wise weighted loss function to enhance reconstruction quality. Specifically, weights are assigned to different layers based on the neural network architecture and are systematically tuned via Bayesian optimization. Experimental results demonstrate that the proposed approximate and weighted attack method outperforms existing state-of-the-art approaches, yielding substantial improvements across multiple image reconstruction metrics.

Index Terms—Data reconstruction attack, federated learning, Bayesian optimization, gradient inversion

arXiv:2308.06822v3 [cs.LG] 15 May 2026

1 Introduction

With the escalating volume of data generated by distributed personal electronic devices, conventional centralized approaches for training machine learning models face substantial obstacles in data collection and privacy preservation. To mitigate these issues, federated learning (FL) [1], [2] has emerged as a promising paradigm.

A prominent feature of FL is that it enables model training on distributed data sources while retaining raw data locally. For instance, in the widely adopted federated averaging (FedAvg) [2] algorithm, each client trains a local model on private data and transmits the updated model parameters to the server. The central server subsequently aggregates these updates to refine a global model. In this manner, FL allows multiple participants to train a robust model without directly exposing raw data, thereby addressing concerns related to data privacy, data security, and data access rights. Consequently, FL has gained significant attention in recent years for addressing growing privacy concerns in applications such as healthcare [3], [4] and cooperative controller learning across autonomous vehicles without sharing historical trajectories [5].

Although model updates in FL were long believed to be safe to share, recent studies [6], [7], [8], [9] have shown that clients' sensitive training data can be compromised through *data reconstruction attacks* [9]. In these attacks, the adversary first randomly initializes dummy samples and labels and then performs forward and backward propagation to obtain dummy model updates. Through an iterative process that minimizes the discrepancy between the dummy model updates and the ground-truth updates, the dummy samples and labels are updated simultaneously.

Existing attack methods have achieved promising results, but attacking FedAvg with multi-step model updates remains challenging. A more detailed discussion of the related literature is provided in Section 2. In particular, existing attacks are significantly less effective in the realistic FedAvg regime with multiple local epochs and multiple mini-batches, where the server observes only the final accumulated model update rather than the hidden epoch-wise updates. This missing-dynamics issue makes the corresponding reconstruction problem largely intractable for existing methods.

To address these challenges, we propose a novel approximate and weighted attack (AWA) method for data reconstruction against FL systems with multi-step model updates. The key contribution of this work is to transform an intractable reconstruction problem in realistic FedAvg settings (multi-epoch and multi-batch) into a tractable one through an interpolation-based approximation of hidden training dynamics. Concretely, we approximate the missing intermediate model updates by interpolating between the leaked final model parameters and the initial model parameters. Furthermore, we enhance the reconstruction process by using a layer-wise weighted loss whose weights are tuned systematically by Bayesian optimization [10].

- *Y. Song is with Division of Mathematical Sciences, School of Physical and Mathematical Sciences, Nanyang Technological University, 21 Nanyang Link, 637371, Singapore. Email: yongcun.song@ntu.edu.sg.*
- *Z. Wang (corresponding author) is with the Chair for Dynamics, Control, Machine Learning and Numerics – Alexander von Humboldt Professorship, Department of Mathematics, Friedrich-Alexander-Universität Erlangen-Nürnberg, Cauerstrasse 11, 91058 Erlangen, Germany. Email: ziqi.wang@fau.de.*
- *E. Zuazua is with the Chair for Dynamics, Control, Machine Learning and Numerics – Alexander von Humboldt Professorship, Department of Mathematics, Friedrich-Alexander-Universität Erlangen-Nürnberg, Cauerstrasse 11, 91058 Erlangen, Germany; the Chair of Computational Mathematics, Fundación Deusto Avda. de las Universidades 24, 48007 Bilbao, Basque Country, Spain; and also with the Departamento de Matemáticas, Universidad Autónoma de Madrid, 28049 Madrid, Spain. Email: enrique.zuazua@fau.de.*

- *Accepted for publication in IEEE Transactions on Big Data.*

Overall, our main contributions are as follows:

1) To attack FL systems with multi-step model updates, we propose an interpolation-based approximation method. The model update associated with each epoch is approximated by interpolating between the initial and updated model parameters, which makes attacks on realistic FedAvg scenarios feasible and effective.
2) To further improve attack performance after approximation, we employ a layer-wise weighted loss function. The layer weights are determined by Bayesian optimization, and we further enhance the weights of layers with large matching errors to improve the attack's adaptability and performance.
3) Our method is architecture-aware through its layer-wise weighting design rather than being tied to a specific network structure. Furthermore, it can reconstruct training data from model updates leaked at different stages of the training process.

The rest of the paper is organized as follows. In Section 2, we review the related literature on data reconstruction attacks in FL. In Section 3, we provide the necessary background on FL and data reconstruction attacks. In Section 4, we analyze data reconstruction under different FedAvg settings. Section 5 presents our proposed AWA method, including the approximation method and the layer-wise weighted loss function. The experimental setup and simulation results are presented in Section 6. Finally, Section 7 concludes the paper.

2 Related Work

Label Inference Techniques. Inferring labels in advance avoids the joint optimization of both samples and labels, thereby markedly simplifying the reconstruction problem. It was first shown in [11] that the label of a single sample can be analytically extracted from the model updates of the last fully connected layer. Later, [8] extended single-sample label extraction to batch settings under the restrictive assumption that labels do not repeat within a batch. This limitation was systematically addressed by the batch label inference approach proposed in [6].

Distance Functions for Matching Updates. To measure the discrepancy between dummy and ground-truth model updates, Euclidean distance is commonly used in the attack loss function; see [8], [9], [11]. Moreover, cosine distance was advocated in [7], [12] because, in high dimensions, the direction of the model updates can carry more information than their magnitude. In [13], a Gaussian kernel based on model-update differences was proposed to measure the discrepancy, allowing the scaling factor in the kernel to adapt to the distribution of model updates in each attack.

Optimizer Choices. Among the optimizers used in these attacks, L-BFGS [14] and Adam [15] are the most common. In particular, the reconstruction performance of these two optimizers was compared in [12], [13], [16]. It was shown in [16] that L-BFGS requires fewer attack iterations than Adam to achieve high reconstruction quality on the LFW dataset [17]. Conversely, [12] reported that L-BFGS obtains lower reconstruction quality than Adam on the CIFAR-10 dataset [18]. Although there is no universal rule for choosing the loss function or optimizer, appropriate selections can substantially improve attack performance in specific scenarios.

Regularization Based on Prior Knowledge. Another way to improve reconstruction performance is to add auxiliary regularization terms to the attack loss function based on prior knowledge of the data. In [16], a label regularizer was proposed to match the dummy samples and labels when both are optimized simultaneously. Moreover, image priors can be incorporated into the reconstruction procedure. For instance, total variation regularization [12] is used to reduce image noise. In [8], the authors proposed a prior term based on the statistical mean and variance stored in the batch normalization layers. In [19], a generative model pretrained on the raw data distribution was used to improve reconstruction. However, the raw data distribution is not necessarily accessible during training.

Attacks on Multi-Step FedAvg Updates. Despite remarkable progress, limited attention has been paid to attacking FedAvg with multi-step updates, where clients share local model parameters only after training for multiple epochs, each executed over multiple mini-batches. GENG [6] extended gradient inversion attacks to FedAvg by modeling the unknown updates along an estimated trajectory between the initial and uploaded client models. However, its setting focuses on full-batch local training and does not consider layer-wise weighting. Another approximation method, AGIC [7], first initializes a combined dummy batch whose size is the sum of all mini-batches used in the client's local training process. This combined dummy batch is then used to perform a single gradient step, and the resulting dummy model update is used to approximate the ground-truth multi-step model update. AGIC employs a weighted loss function, but the layer weights are chosen empirically rather than systematically.

3 Preliminaries

In this section, we first describe the mathematical formulation and training process of FL and then introduce the formulation and setup of data reconstruction attacks.

3.1 Problem Statement of FL

In FL, the training data are distributed across K clients, and the global training objective can be formulated as the following minimization problem [1]:

$$\min_{\theta \in \mathbb{R}^d} \sum_{k=1}^{K} \rho_k F_k(\theta), \tag{1}$$

where F_k is the local loss function of client k with model parameters θ, and $\rho_k \geq 0$ with $\sum_{k=1}^{K} \rho_k = 1$ specifies the relative weight of client k. In practice, F_k is typically defined as the empirical risk over client k's local dataset $\{(x_i^{(k)}, y_i^{(k)})\}_{i=1}^{N^{(k)}}$ of size $N^{(k)}$, i.e., $F_k(\theta) = 1/N^{(k)} \sum_{i=1}^{N^{(k)}} \ell(x_i^{(k)}, y_i^{(k)}; \theta)$, where ℓ denotes the loss for a given training sample. Common choices include the ℓ_2 loss and the cross-entropy loss, see [20] for more options. The relative weight ρ_k is often chosen as $\rho_k = N^{(k)}/N_K$, where $N_K = \sum_{k=1}^{K} N^{(k)}$ is the total size of all clients' datasets.

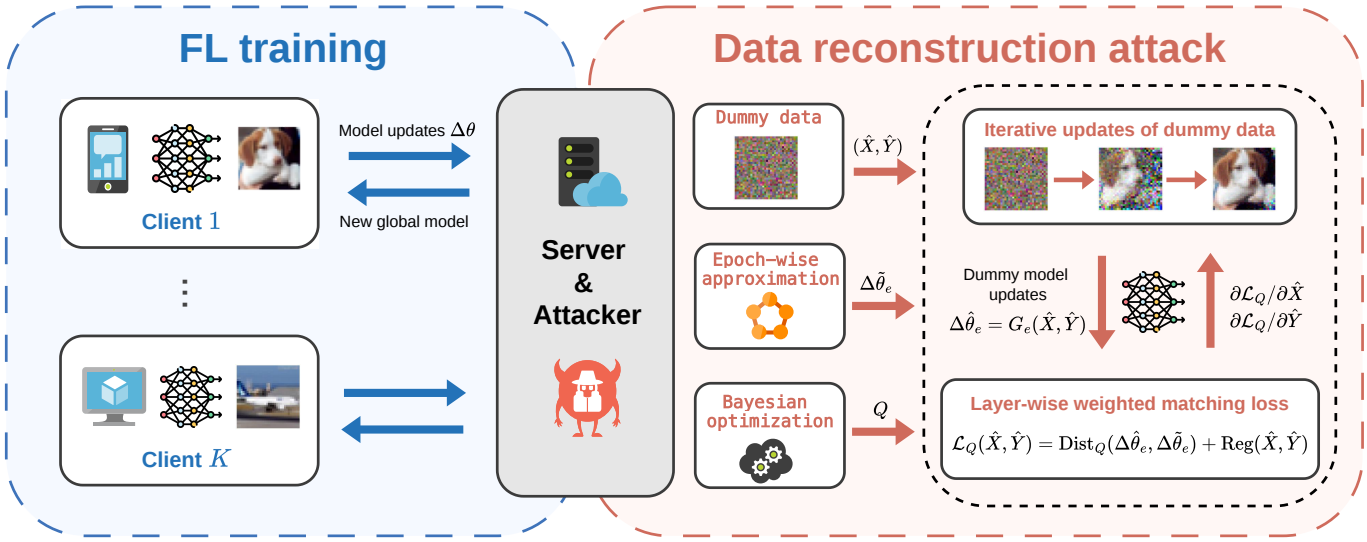


Figure 1: Illustration of the FL training process and the approximate and weighted attack process.

3.2 FedAvg Algorithm

To solve (1), FL algorithms normally involve a sequence of training rounds in which the clients perform local updates and the server performs aggregation. To fix ideas, we focus on FedAvg [2], which is the most commonly used algorithm in FL.

In FedAvg, at each round t, the server selects a subset $\mathcal{K}_t$ of clients to participate and sends them the current global model parameters θ_t. Each selected client $k \in \mathcal{K}_t$ then sets its local model parameters $\theta_t^{(k)} = \theta_t$ and updates $\theta_t^{(k)}$ for E epochs, each consisting of B mini-batches. After training, client k's model update $\Delta\theta_t^{(k)}$ can be written as

$$\Delta\theta_t^{(k)} = -\eta \sum_{e=1}^{E} \sum_{b=1}^{B} \nabla_{\theta_{t,e,b}^{(k)}} \ell\left(X_{t,e,b}^{(k)}, Y_{t,e,b}^{(k)}\right), \tag{2}$$

where $\ell(X_{t,e,b}^{(k)}, Y_{t,e,b}^{(k)})$ denotes the loss evaluated on the mini-batch $(X_{t,e,b}^{(k)}, Y_{t,e,b}^{(k)})$ with respect to the model parameters $\theta_{t,e,b}^{(k)}$. The subscripts t, e, and b index the round, epoch, and mini-batch, respectively. Consequently, client k's local model parameters are updated to

$$\theta_{t+1}^{(k)} = \theta_t^{(k)} + \Delta\theta_t^{(k)}. \tag{3}$$

Finally, client k sends its updated local model parameters $\theta_{t+1}^{(k)}$ back to the server for averaging.

3.3 Vanilla Data Reconstruction Attack

Although clients share only updated model parameters with the server, their private training data remain vulnerable to data reconstruction attacks. In this subsection, we introduce the formulation and general procedure of a data reconstruction attack.

For notational simplicity, we consider an arbitrary attack on client k at round t. We then omit the client and round indices (k, t) whenever no ambiguity arises.

As shown in (2), during the local training process, the model update $\Delta\theta$ consists of gradients computed over $E \times B$ mini-batches. Let G denote the map from the training data (X, Y) to the model update $\Delta\theta$. Then (2) can be written compactly as

$$\Delta\theta = G(X, Y). \tag{4}$$

For an attacker with access to $\Delta\theta$, reconstructing (X, Y) is essentially an inverse problem. In particular, if G^{-1} exists and is known analytically, the attacker can recover (X, Y) directly as follows:

$$(X, Y) = G^{-1}(\Delta\theta). \tag{5}$$

However, since neural networks are highly nonlinear and the model updates are aggregated over multiple mini-batches, it is generally difficult to identify G^{-1}. To address this, we introduce a numerical approach for solving (4).

Assuming that the attacker knows the client's training procedure and can therefore evaluate G, problem (4) can be approached numerically as follows. First, to launch the attack, the attacker randomly initializes a dummy dataset $(\hat{X}, \hat{Y})$ with the same dimensions as the client's ground-truth dataset (X, Y). The attacker then uses G to calculate the dummy model update $\Delta\hat{\theta}$ given by

$$\Delta\hat{\theta} = G(\hat{X}, \hat{Y}). \tag{6}$$

Then, the attacker reconstructs the client's data by minimizing the following loss function:

$$\mathcal{L}_m(\hat{X}, \hat{Y}) = \mathrm{Dist}(\Delta\hat{\theta}, \Delta\theta) + \mathrm{Reg}(\hat{X}, \hat{Y}). \tag{7}$$

Here, Reg denotes regularization terms that improve reconstruction quality, such as total variation regularization [12] or batch normalization regularization [8]. The Dist term quantifies the discrepancy between the dummy and true model updates. Common choices include the ℓ_2 distance [11] and cosine distance [7], with the latter defined as

$$\|\Delta\hat{\theta} - \Delta\theta\|_{\cos} = 1 - \frac{\langle \Delta\hat{\theta}, \Delta\theta \rangle}{\|\Delta\hat{\theta}\| \, \|\Delta\theta\|}. \tag{8}$$

Finally, the reconstructed data $(\hat{X}^*, \hat{Y}^*)$ can be obtained by solving the following optimization problem:

$$(\hat{X}^*, \hat{Y}^*) = \arg\min_{\hat{X}, \hat{Y}} \mathcal{L}_m(\hat{X}, \hat{Y}). \tag{9}$$

The iterative updates of $\hat{X}$ and $\hat{Y}$ can be carried out using gradient descent with learning rate $\hat{\eta}$:

$$\hat{X}^+ = \hat{X} - \hat{\eta}\nabla_{\hat{X}}\mathcal{L}_m(\hat{X}, \hat{Y}),$$
$$\hat{Y}^+ = \hat{Y} - \hat{\eta}\nabla_{\hat{Y}}\mathcal{L}_m(\hat{X}, \hat{Y}).$$

Remark 3.1 (An honest-but-curious server (attacker))**.** *A data reconstruction attacker needs to evaluate the mapping G in* (6)*, which encapsulates the client's local training procedure. Without knowledge of the local hyperparameters, reproducing G, or equivalently replicating the client's local training, is generally infeasible. However, in practice, the central server holds substantial information about the training process. Consequently, an honest-but-curious server can conduct data reconstruction attacks when it has access to the following information:*

1) *Model architecture: The server normally determines the neural network architecture shared by all clients.*
2) *Initial model parameters: The client's initial local model parameters θ are dispatched by the server.*
3) *Model update: Each client sends the updated model parameters θ^+ back to the server. Thus, the server can readily obtain the model update $\Delta\theta = \theta^+ - \theta$.*
4) *Loss function: The server typically also knows the form of the loss function ℓ shared by all clients. This choice usually remains unchanged during training.*
5) *Dataset size: This information is shared with the server for weighted aggregation.*
6) *Client's local training hyperparameters: The server can assign hyperparameters such as the number of epochs E, the number of mini-batches B, and the learning rate η to control the training workload and procedure.*

However, unless additional leakage is assumed, the attacker does *not* observe the client's random reshuffling of the dataset at each epoch, the resulting mini-batch partitions, or the intermediate local model parameters. Therefore, the attacker's knowledge is fixed, and the differences among the four scenarios discussed below lie in whether the information visible to the server is sufficient to evaluate G exactly.

4 Analysis of Data Reconstruction under Different FedAvg Settings

In this section, we compare four FedAvg scenarios determined by the number of local epochs E and the number of mini-batches B. The adversary is always the same honest-but-curious server (attacker) described in Remark 3.1. Hence, these four scenarios are not distinguished by different attacker capabilities, but rather by whether the information available to the attacker is sufficient to evaluate the client's local training map G in (6) and thus calculate the dummy model update.

Scenario 1: $E = 1, B = 1$**.** In this scenario, the attacker can evaluate G exactly and replicate the client's training process. Indeed, since $E = 1$ and $B = 1$, the client uses full-batch gradient descent for one epoch as follows:

$$G(X, Y) = -\eta\nabla_\theta \ell\left(X, Y\right). \tag{10}$$

Since the update is computed from a single full batch at the known parameter θ, no hidden shuffling or intermediate state is involved. Thus, the attacker can replicate the client's training process by replacing (X, Y) with the dummy dataset $(\hat{X}, \hat{Y})$.

Scenario 2: $E > 1, B = 1$**.** The attacker can also evaluate G exactly in this scenario. Specifically, the client uses full-batch gradient descent for E epochs:

$$G(X, Y) = -\eta \sum_{e=1}^{E} \nabla_{\theta_e} \ell\left(X, Y\right). \tag{11}$$

Although there are multiple epochs, every epoch still uses the whole dataset since $B = 1$. Hence, there is no hidden mini-batch partition to infer, and the attacker can roll out exactly the same sequence of intermediate parameters when replacing (X, Y) with the dummy dataset $(\hat{X}, \hat{Y})$ and training the model for E epochs. Therefore, Scenario 2 remains exactly reproducible.

Scenario 3: $E = 1, B > 1$**.** In this scenario, since $B > 1$, the client uses mini-batch gradient descent for one epoch in the following way:

$$G(X, Y) = -\eta \sum_{b=1}^{B} \nabla_{\theta_b} \ell\left(X_b, Y_b\right). \tag{12}$$

The key point is that the hidden randomness occurs only once because $E = 1$. Consequently, the dataset is shuffled only once. The attacker can first partition its dummy dataset $(\hat{X}, \hat{Y})$ into B mini-batches $\{(\hat{X}_b, \hat{Y}_b)\}_{b=1}^{B}$. Then, by replacing (X_b, Y_b) in (12) with $(\hat{X}_b, \hat{Y}_b)$, it can reproduce the client's training process. Therefore, Scenario 3 remains tractable for existing reconstruction attacks.

Scenario 4: $E > 1, B > 1$**.** In this scenario, the attacker cannot evaluate the exact map G needed to calculate the dummy model update $\Delta\hat{\theta}$ defined in (6). More concretely, the client uses mini-batch gradient descent for E epochs:

$$G(X, Y) = -\eta \sum_{e=1}^{E} \sum_{b=1}^{B} \nabla_{\theta_{e,b}} \ell\left(X_{e,b}, Y_{e,b}\right). \tag{13}$$

In each epoch, the client first reshuffles its dataset and then partitions it into B mini-batches. Consequently, the final update depends on a sequence of hidden mini-batch partitions across epochs. However, information about this epoch-wise shuffling is never available to the attacker. If the attacker simply performs E epochs of updates with arbitrarily chosen mini-batch orders, the resulting dummy training is no longer guaranteed to match the client's original training process.

Overall, while existing attack methods are generally applicable to Scenarios 1–3, the substantially more challenging Scenario 4 has received limited attention. To overcome the difficulties inherent to this scenario, we propose an interpolation-based approximation method. By interpolating between the initial and the final updated model parameters, the attacker can effectively estimate the hidden intermediate model updates for each epoch. This approximation reduces the intractable Scenario 4 to independent instances of the tractable Scenario 3. A detailed formulation of the proposed method is presented in the subsequent section.

5 Approximate and Weighted Attack in Data Reconstruction

In this section, we propose an approximate and weighted data reconstruction attack. The algorithm is summarized in Algorithm 1 and illustrated in Figure 1.

5.1 Approximation of the Intermediate Model Updates

As discussed in Section 4, when $E > 1$ and $B > 1$, reshuffling the dataset at each epoch makes it difficult for the attacker to reproduce G. However, if the attacker knew the intermediate model update $\Delta\theta_e$ for any epoch $e \in \{1, 2, \ldots, E\}$, the problem could be reduced from Scenario 4 ($E > 1$ and $B > 1$) to Scenario 3 ($E = 1$ and $B > 1$) by attacking each epoch separately.

To this end, we interpolate between the updated model parameters θ^+ and the initial model parameters θ to approximate the intermediate model parameters at the end of each epoch. In particular, for a client that trains its model for E epochs, the approximate intermediate model parameters $\{\tilde{\theta}_e\}_{e=1}^E$ are given by

$$\tilde{\theta}_e = \theta + \frac{e}{E}\left(\theta^+ - \theta\right), \quad e = 1, 2, \ldots, E. \tag{14}$$

Then, the approximate model updates $\{\Delta\tilde{\theta}_e\}_{e=1}^E$ corresponding to each epoch can be obtained as

$$\Delta\tilde{\theta}_e = \tilde{\theta}_e - \tilde{\theta}_{e-1}, \quad e = 1, 2, \ldots, E, \tag{15}$$

where $\tilde{\theta}_0 = \theta$ is the initial model parameters of the round.

After obtaining the approximate updates $\{\Delta\tilde{\theta}_e\}_{e=1}^E$, the attacker can use them as surrogates for the unknown intermediate model updates $\Delta\theta_e$. In other words, the attack problem is reduced from Scenario 4 ($E > 1$ and $B > 1$) to a one-epoch Scenario 3 problem ($E = 1$ and $B > 1$).

We denote by G_e the client's training process at epoch e. Then, the dummy model update $\Delta\hat{\theta}_e$ can be obtained according to (12) as

$$\Delta\hat{\theta}_e = G_e(\hat{X}, \hat{Y}) = -\eta \sum_{b=1}^{B} \nabla_{\theta_{e,b}} \ell\left(\hat{X}_{e,b}, \hat{Y}_{e,b}\right). \tag{16}$$

Finally, the attack can be carried out by following the procedure of Scenario 3. The same idea also applies when each epoch contains only one mini-batch. By leveraging interpolation, the attacker can reconstruct the data by training for a single epoch using the approximate model update, which substantially reduces computational overhead, especially when E is large. Under linear interpolation, the approximate epoch-wise update $\Delta\tilde{\theta}_e$ is identical for all e. This approximation is reasonable, particularly when training involves many epochs and a small learning rate. Its effectiveness is demonstrated through extensive experiments in Section 6. Other interpolation schemes may also be considered and deserve further investigation.

5.2 Improved Weighted Loss Function for the Data Reconstruction Attack

The commonly used loss function (7) for data reconstruction treats different components of the model update $\Delta\tilde{\theta}_e$ equally. However, as observed in [21], different layers in a neural network contribute differently to performance. Motivated by this observation, we assign different weights to different layers to facilitate reconstruction. The construction of our method is described below.

Consider a neural network with L layers. The model update $\Delta\tilde{\theta}_e$ can be decomposed into layer-wise updates as $\Delta\tilde{\theta}_e = \{\Delta\tilde{\theta}_e^{(l)}\}_{l=1}^L$. Similarly, the dummy model update can be written as $\Delta\hat{\theta}_e = \{\Delta\hat{\theta}_e^{(l)}\}_{l=1}^L$. By assigning a positive weight $q^{(l)}$ to layer l, the loss function in (7) is modified as

$$\mathcal{L}_Q(\hat{X}, \hat{Y}) = \mathrm{Dist}_Q(\Delta\hat{\theta}_e, \Delta\tilde{\theta}_e) + \mathrm{Reg}(\hat{X}, \hat{Y}). \tag{17}$$

Here, when cosine distance is used, Dist_Q is defined as

$$1 - \frac{\sum_{l=1}^L q^{(l)} \langle \Delta\hat{\theta}_e^{(l)}, \Delta\tilde{\theta}_e^{(l)} \rangle}{\sqrt{\sum_{l=1}^L q^{(l)} \|\Delta\hat{\theta}_e^{(l)}\|_2^2} \sqrt{\sum_{l=1}^L q^{(l)} \|\Delta\tilde{\theta}_e^{(l)}\|_2^2}}. \tag{18}$$

The weighted loss function leverages the distinct characteristics of different layers in the model update. In the next subsection, we introduce a systematic method for designing the layer weights to enhance reconstruction performance.

5.2.1 Design of Layer Weights

Increasing weights layer by layer. We consider linearly increasing weight profiles for different types of layers. To illustrate the idea clearly, we focus on the commonly used ResNet architecture [22], which contains convolutional, batch normalization, and fully connected layers. Specifically, we design the following weight functions for each type of layer:

$$q_{\mathrm{cv}}^{(l)} = \begin{cases} \frac{q_{\mathrm{cv}}-1}{L_{\mathrm{cv}}-1}(l-1)+1, & 1 \le l \le L_{\mathrm{cv}},\ L_{\mathrm{cv}} > 1, \\ q_{\mathrm{cv}}, & l = L_{\mathrm{cv}} = 1, \end{cases} \tag{19a}$$

$$q_{\mathrm{bn}}^{(l)} = \begin{cases} \frac{q_{\mathrm{bn}}-1}{L_{\mathrm{bn}}-1}(l-1)+1, & 1 \le l \le L_{\mathrm{bn}},\ L_{\mathrm{bn}} > 1, \\ q_{\mathrm{bn}}, & l = L_{\mathrm{bn}} = 1, \end{cases} \tag{19b}$$

$$q_{\mathrm{fc}}^{(l)} = \begin{cases} \frac{q_{\mathrm{fc}}-1}{L_{\mathrm{fc}}-1}(l-1)+1, & 1 \le l \le L_{\mathrm{fc}},\ L_{\mathrm{fc}} > 1, \\ q_{\mathrm{fc}}, & l = L_{\mathrm{fc}} = 1, \end{cases} \tag{19c}$$

where L_{cv}, L_{bn}, and L_{fc} denote the numbers of convolutional, batch normalization, and fully connected layers, respectively, and $L = L_{\mathrm{cv}} + L_{\mathrm{bn}} + L_{\mathrm{fc}}$. The values $q_{\mathrm{cv}} > 1$, $q_{\mathrm{bn}} > 1$, and $q_{\mathrm{fc}} > 1$ are the maximum weights assigned to the last layer of each respective type. For a given neural network with a fixed number of layers (L_{cv}, L_{bn}, and L_{fc}), the values of q_{cv}, q_{bn}, and q_{fc} determine the slopes of the linearly increasing weight functions in (19).

Enhancing the weights of layers with larger errors. Applying the linearly increasing weights determined by (19) in the loss function (17) may overemphasize certain layers and lead to biased reconstructions. To balance the benefits of linearly increasing weights against this risk, we modify the weights of layers with larger errors by exploiting statistical information, namely the mean $\mu(\cdot)$ and variance $\sigma^2(\cdot)$ of the layer-wise model updates. The enhancement procedure is described below.

First, we compute the relative errors $e_{\mathrm{mean}}^{(l)}$ and $e_{\mathrm{var}}^{(l)}$ between the dummy model update $\Delta\hat{\theta}_e^{(l)}$ and the target approximate model update $\Delta\tilde{\theta}_e^{(l)}$ at each layer as follows:

$$e_{\mathrm{mean}}^{(l)} = \frac{|\mu(\Delta\hat{\theta}_e^{(l)}) - \mu(\Delta\tilde{\theta}_e^{(l)})|}{|\mu(\Delta\tilde{\theta}_e^{(l)})|}, \quad l = 1, 2, \ldots, L, \tag{20}$$

$$e_{\text{var}}^{(l)} = \frac{|\sigma^2(\Delta\hat{\theta}_e^{(l)}) - \sigma^2(\Delta\tilde{\theta}_e^{(l)})|}{|\sigma^2(\Delta\tilde{\theta}_e^{(l)})|}, \quad l = 1, 2, \ldots, L. \tag{21}$$

Next, we select a subset $\mathcal{P} \subseteq [L] := \{1, 2, \ldots, L\}$ of layers with the largest relative errors according to $\{e_{\text{mean}}^{(l)}\}_{l=1}^{L}$ and $\{e_{\text{var}}^{(l)}\}_{l=1}^{L}$. For these layers, we replace their original weights with a fixed value q_{en}, by setting

$$q^{(l)} = q_{\text{en}}, \quad l \in \mathcal{P}. \tag{22}$$

Here, $q_{\text{en}} > 1$ is a hyperparameter to be determined. This replacement strategy, rather than multiplying the existing weights by another constant, prevents assigning excessively large weights in deeper layers, which could lead to instability or degradation of reconstruction performance.

The choice of the subset $\mathcal{P}$ is determined by the proportional parameters $p_{\text{mean}} \in [0, 1]$ and $p_{\text{var}} \in [0, 1]$. For a given p_{mean}, we first select $N_{\text{mean}} = \lceil p_{\text{mean}} \cdot L \rceil$ layers with the largest relative errors in $\{e_{\text{mean}}^{(l)}\}_{l=1}^{L}$. Let the set of indices corresponding to the N_{mean} layers be denoted as $\mathcal{P}_{\text{mean}}$, that is

$$\mathcal{P}_{\text{mean}} = \{i_1, i_2, \ldots, i_{N_{\text{mean}}}\}, \tag{23}$$

where $i_1, i_2, \ldots, i_{N_{\text{mean}}} \in [L]$ are selected such that $e_{\text{mean}}^{(i_1)} \geq e_{\text{mean}}^{(i_2)} \geq \ldots \geq e_{\text{mean}}^{(i_{N_{\text{mean}}})} \geq e_{\text{mean}}^{(l)}$ for all $l \in [L] \setminus \mathcal{P}_{\text{mean}}$.

Similarly, we obtain a set $\mathcal{P}_{\text{var}}$ with $N_{\text{var}} = \lceil p_{\text{var}} \cdot L \rceil$ elements as

$$\mathcal{P}_{\text{var}} = \{j_1, j_2, \ldots, j_{N_{\text{var}}}\}, \tag{24}$$

where $j_1, j_2, \ldots, j_{N_{\text{var}}} \in [L]$ are selected such that $e_{\text{var}}^{(j_1)} \geq e_{\text{var}}^{(j_2)} \geq \ldots \geq e_{\text{var}}^{(j_{N_{\text{var}}})} \geq e_{\text{var}}^{(l)}$ for all $l \in [L] \setminus \mathcal{P}_{\text{var}}$.

Finally, the subset $\mathcal{P}$ can be obtained as the intersection of $\mathcal{P}_{\text{mean}}$ and $\mathcal{P}_{\text{var}}$:

$$\mathcal{P} = \mathcal{P}_{\text{mean}} \cap \mathcal{P}_{\text{var}}. \tag{25}$$

Hyperparameters to tune. Following (19) and (22), the layer weights $\{q^{(l)}\}_{l=1}^{L}$ in (17) are determined by the parameter vector $Q \in \mathbb{R}^6$, defined as

$$Q = (q_{\text{cv}}, q_{\text{bn}}, q_{\text{fc}}, q_{\text{en}}, p_{\text{mean}}, p_{\text{var}}). \tag{26}$$

Given Q, the reconstructed data $(\hat{X}^*, \hat{Y}^*)$ are obtained by solving the following optimization problem:

$$(\hat{X}^*, \hat{Y}^*) = \arg\min_{\hat{X}, \hat{Y}} \mathcal{L}_Q(\hat{X}, \hat{Y}). \tag{27}$$

We then use Bayesian optimization to choose an appropriate Q for better reconstruction.

5.2.2 Choice of Q by Bayesian Optimization

Objective function. As shown in (27), for a given Q one can obtain the reconstructed data $(\hat{X}^*, \hat{Y}^*)$. Then, the corresponding reconstructed model update can be computed as $\Delta\hat{\theta}_e^* = G_e(\hat{X}^*, \hat{Y}^*)$.

Let $f : \mathbb{R}^6 \to \mathbb{R}$ be the objective function that quantifies the discrepancy between the reconstructed model update $\Delta\hat{\theta}_e^*$ and the target approximate model update $\Delta\tilde{\theta}_e$. Specifically, we use cosine distance as the metric:

$$f(Q) = \|\Delta\hat{\theta}_e^* - \Delta\tilde{\theta}_e\|_{\cos}. \tag{28}$$

Finding the optimal Q^* is equivalent to solving the following optimization problem:

$$Q^* = \arg\min_{Q} f(Q). \tag{29}$$

For this optimization problem, f is a black-box function that does not admit an analytic expression. Moreover, evaluating f is computationally expensive because each evaluation requires a full data reconstruction attack to obtain $\Delta\hat{\theta}_e^*$. As a result, traditional parameter-selection methods such as grid search are not feasible. To overcome these difficulties, we employ Bayesian optimization [10] to solve (29).

A Bayesian optimization algorithm for (29)**.** Bayesian optimization is a powerful technique for optimizing black-box functions that are expensive to evaluate and may involve noise or other sources of uncertainty. In general, Bayesian optimization iteratively uses a surrogate model to approximate the black-box function and then employs an acquisition function to determine the next set of parameters to evaluate.

The surrogate model approximates the black-box objective function f and is commonly chosen to be a Gaussian process (GP) [23]. Formally, a GP is a collection of random variables, any finite number of which have a joint Gaussian distribution.

Given an initial set $\mathcal{O} = \{(Q_i, f(Q_i))\}_{i=1}^{n}$ that contains n sampled points and their function values, the resulting prior distribution is

$$\boldsymbol{f} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma_{\boldsymbol{QQ}}), \tag{30}$$

where $\boldsymbol{f} = (f(Q_1), \ldots, f(Q_n))^\top$, $\boldsymbol{Q} = (Q_1, \ldots, Q_n)^\top$, and $\boldsymbol{\mu} = (\mu(Q_1), \ldots, \mu(Q_n))^\top$, with $\mu(\cdot)$ denoting the mean function, which is commonly set to zero. Moreover, $\Sigma_{\boldsymbol{QQ}} \in \mathbb{R}^{n \times n}$ is the matrix of covariances with its (i, j) entry given by $\kappa(Q_i, Q_j)$, and $\kappa(\cdot, \cdot)$ is a positive definite kernel function, typically chosen as the Gaussian kernel.

Then, we can infer the value of $f(Q)$ at a new point Q by computing the posterior distribution of $f(Q)$ given prior observations [23] as follows:

$$\begin{aligned} f(Q) \mid \boldsymbol{f} &\sim \mathcal{N}(\mu_Q, \sigma_Q^2), \\ \mu_Q &= \Sigma_{Q\boldsymbol{Q}} \Sigma_{\boldsymbol{QQ}}^{-1} \boldsymbol{f}, \\ \sigma_Q^2 &= \Sigma_{QQ} - \Sigma_{Q\boldsymbol{Q}} \Sigma_{\boldsymbol{QQ}}^{-1} \Sigma_{\boldsymbol{Q}Q}. \end{aligned} \tag{31}$$

The acquisition function is used to propose the parameter vector for the next trial by balancing exploitation and exploration. Exploitation means sampling at locations where the surrogate model predicts a low objective value, whereas exploration means sampling at locations where the predictive uncertainty is high.

One of the most popular acquisition functions is expected improvement (EI) [24]. Let $f_{\min}$ denote the best function value obtained so far. Then, the improvement over $f_{\min}$ at point Q can be defined as

$$I(Q) = \max(f_{\min} - f(Q), 0). \tag{32}$$

The improvement $I(Q)$ is a random variable since $f(Q) \sim \mathcal{N}\left(\mu_Q, \sigma_Q^2\right)$ as shown in (31). To obtain the expected improvement, we can take the expected value as follows:

$$EI(Q) = \mathbb{E}[\max(f_{\min} - f(Q), 0)]. \tag{33}$$

The expected improvement can be evaluated analytically under the GP [25] and is given by

$$EI(Q) = (f_{\min} - \mu_Q)\,\Phi\left(\frac{f_{\min} - \mu_Q}{\sigma_Q}\right) + \sigma_Q \varphi\left(\frac{f_{\min} - \mu_Q}{\sigma_Q}\right), \tag{34}$$

where φ and Φ are the probability density and cumulative distribution functions of the standard normal distribution, respectively. Thus, $EI(Q)$ is higher for a point Q predicted to have a smaller μ_Q and a larger σ_Q, indicating the trade-off between exploitation and exploration.

Given the EI, the parameter Q_{n+1} for the next trial is chosen to be the one with the largest expected improvement:

$$Q_{n+1} = \arg\max_{Q} EI(Q). \tag{35}$$

Evaluating $EI(Q)$ is much easier than evaluating the function f in (29). The optimization problem (35) can be solved using classical techniques such as Newton's method.

Finally, based on the above discussion, we summarize our approximate and weighted data reconstruction attack method in Algorithm 1.

Algorithm 1 Approximate and Weighted Attack (AWA).

Intercept a client's model update $\Delta\theta$.
Approximate the epoch-wise $\Delta\tilde{\theta}_e$ and G_e. ▷ (15)
Initialize an empty set $\mathcal{O}$.
for each trial $i = 1, 2, \ldots, n$ **do**
  Generate a random test point Q_i.
  Obtain $\hat{X}$, $\hat{Y}$, and $f(Q_i)$ via **RecAttack**$(Q_i, \Delta\tilde{\theta}_e, G_e)$.
  Update $\mathcal{O} \leftarrow \mathcal{O} \cup \{(Q_i, f(Q_i))\}$.
end for
for each trial $i = n+1, n+2, \ldots, N_{\mathrm{BO}}$ **do**
  Fit a GP of f with the samples in $\mathcal{O}$. ▷ (30)
  Choose Q_i with the largest EI. ▷ (35)
  Obtain $\hat{X}$, $\hat{Y}$, and $f(Q_i)$ via **RecAttack**$(Q_i, \Delta\tilde{\theta}_e, G_e)$.
  Update $\mathcal{O} \leftarrow \mathcal{O} \cup \{(Q_i, f(Q_i))\}$.
end for
Set $Q^* = \arg\min_{Q_i}\{f(Q_i)\}_{i=1}^{N_{\mathrm{BO}}}$.
Obtain $\hat{X}^*$, $\hat{Y}^*$ via **RecAttack**$(Q^*, \Delta\tilde{\theta}_e, G_e)$.
return $\hat{X}^*$, $\hat{Y}^*$.

RecAttack$(Q_i, \Delta\tilde{\theta}_e, G_e)$:
Initialize the dummy data $(\hat{X}, \hat{Y})$ and set $Q = Q_i$.
for each attack iteration from 1 to N_{AT} **do**
  Compute $\Delta\hat{\theta}_e = G_e(\hat{X}, \hat{Y})$.
  Calculate $\{q^{(l)}\}_{l=1}^{L}$ based on Q. ▷ (19)
  Select layers $l \in \mathcal{P}$ with the largest errors. ▷ (25)
  Set $q^{(l)} = q_{\mathrm{en}}$ for all $l \in \mathcal{P}$ based on Q. ▷ (22)
  Calculate the weighted loss $\mathcal{L}_Q(\hat{X}, \hat{Y})$. ▷ (17)
  Update $\hat{X}^+ = \hat{X} - \hat{\eta}\nabla_{\hat{X}}\mathcal{L}_Q(\hat{X}, \hat{Y})$.
  Update $\hat{Y}^+ = \hat{Y} - \hat{\eta}\nabla_{\hat{Y}}\mathcal{L}_Q(\hat{X}, \hat{Y})$.
  Set $\hat{X} = \hat{X}^+$ and $\hat{Y} = \hat{Y}^+$.
end for
Calculate $f(Q) = \|G_e(\hat{X}, \hat{Y}) - \Delta\tilde{\theta}_e\|_{\cos}$.
return $\hat{X}$, $\hat{Y}$, $f(Q)$.

6 Numerical Experiments

In this section, we report numerical results. We first describe the experimental setup and implementation details. We then explain the choice of hyperparameters and evaluation metrics. Finally, we test AWA on image data reconstruction and compare its performance with three state-of-the-art methods, namely iDLG [11], AGIC [7], and GENG [6], across various scenarios. The source code for our AWA method is available at https://github.com/DCN-FAU-AvH/FL-AWA-attack.

6.1 Experimental Setup

Hardware. All experiments were conducted on a cluster node at FAU equipped with AMD EPYC 7662 CPUs and NVIDIA A100/V100 GPUs.

Implementation details. In our implementation, we use images from ImageNet [26] (color images from 1,000 categories, each of size 224×224) as the clients' ground-truth training data. The model used by each client is ResNet18 [22]. The client's local training process uses stochastic gradient descent with a learning rate of 10^{-4}. For attack optimization, we use the Adam optimizer [15] with a learning rate of 0.1, together with the cosine distance loss defined in (8), total variation regularization, and batch normalization regularization. Following the label inference methods in [6], [8], we assume that the labels are known. Table 1 lists the simulation settings for the data reconstruction attacks.

TABLE 1: Simulation settings for the attacks.

Client's dataset	ImageNet
Client's neural network	ResNet18
Attack optimizer	Adam
Attack learning rate	0.1
Attack iterations	10,000

As discussed in Section 4, challenging FedAvg scenarios with $E > 1$ and $B > 1$ require an approximation strategy. For iDLG, we adapt the original approach by using random mini-batch orders across epochs. Our AWA method leverages (15) to obtain approximate intermediate model updates and reconstruct the client's data using the first approximate epoch-wise update. GENG adopts a similar interpolation idea. However, it does not address the scenario with multiple mini-batches and lacks a layer-wise weighting mechanism. AGIC approximates the received model update by performing a single step of full-batch gradient descent on a combined batch that aggregates all mini-batches used in the client's local training (of size $N \times E$), which may lead to excessively high-dimensional dummy inputs during optimization. In all experiments, iDLG and GENG employ an unweighted loss function, whereas AGIC uses a weighted loss function with empirically chosen weights for convolutional and fully connected layers. In contrast, our AWA method uses an enhanced weighted loss whose layer weights are determined by Bayesian optimization.

Evaluation metrics. To evaluate attack effectiveness and reconstruction quality, we employ three widely used metrics that quantify the difference between the reconstructed data and the ground-truth data: Learned Perceptual Image Patch Similarity (LPIPS) [27], Peak Signal-to-Noise Ratio (PSNR)

[28], and the Structural Similarity Index Measure (SSIM) [29]. These metrics are standard and well-established indicators of image reconstruction quality and have been widely adopted in evaluations of data reconstruction attacks, see [6], [7], [12], [16].

6.2 Parameter Tuning via Bayesian Optimization

In our AWA method, the parameter vector Q defined in (26) is optimized via Bayesian optimization. Specifically, we employ a Gaussian process surrogate model as described in (31) and use the expected improvement criterion (34) as the acquisition function. The Bayesian optimization objective is the cosine distance between the reconstructed model update and the target model update, as defined in (28).

Instead of tuning Q for each attack, we perform the optimization once in a moderate-scale scenario ($N = 16$, $E = 32$, $B = 4$) and subsequently apply the optimized Q^* across all scenarios. Our numerical results in Table 3 demonstrate that this strategy consistently yields improved performance in both small- and large-scale scenarios while significantly reducing computational overhead.

Specifically, we perform Bayesian optimization for $N_{\text{BO}} = 50$ iterations, with the first $n = 12$ iterations used to initialize the observation set $\mathcal{O}$. The search ranges and the optimized values for each component of Q are summarized in Table 2. The cumulative minimum loss $f(Q)$ over 50 trials is shown in Figure 2, illustrating that Bayesian optimization consistently discovers lower $f(Q)$ values as the number of trials increases. The final tuned parameters obtained by Bayesian optimization for our AWA method are reported in the last column of Table 2.

TABLE 2: Search ranges and tuned values of Q obtained by Bayesian optimization.

Parameter	Search range	Tuned value
q_{cv}	[1, 1000]	694.99
q_{bn}	[1, 1000]	898.67
q_{fc}	[1, 1000]	441.64
q_{en}	[1, 1000]	789.51
p_{mean}	[0, 0.5]	0.497
p_{var}	[0, 0.5]	0.314

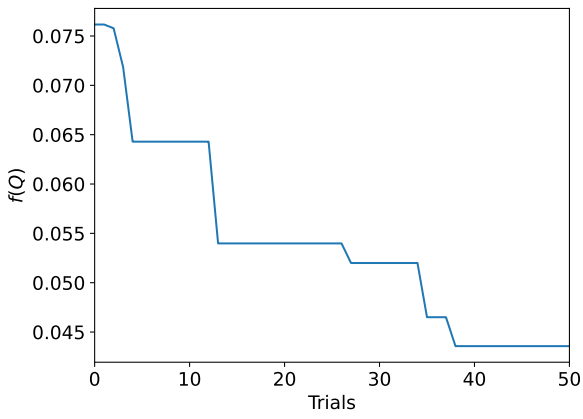
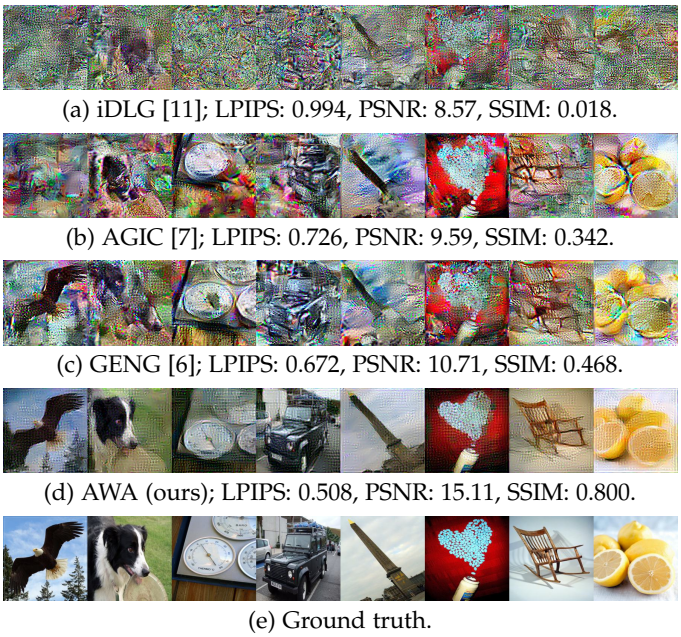


Figure 2: Cumulative minimum of Bayesian optimization.



(a) iDLG [11]; LPIPS: 0.994, PSNR: 8.57, SSIM: 0.018.

(b) AGIC [7]; LPIPS: 0.726, PSNR: 9.59, SSIM: 0.342.

(c) GENG [6]; LPIPS: 0.672, PSNR: 10.71, SSIM: 0.468.

(d) AWA (ours); LPIPS: 0.508, PSNR: 15.11, SSIM: 0.800.

(e) Ground truth.

Figure 3: Comparison results for attacking a FedAvg scenario ($N = 8$, $E = 32$, $B = 1$). More scenarios and quantitative metrics are summarized in Table 3.

6.3 Results of the Image Data Reconstruction Attacks

To verify the feasibility and effectiveness of our proposed AWA method, we evaluate it on ImageNet-based data reconstruction attacks and compare it with state-of-the-art methods, including iDLG [11], AGIC [7], and GENG [6], under various FedAvg scenarios.

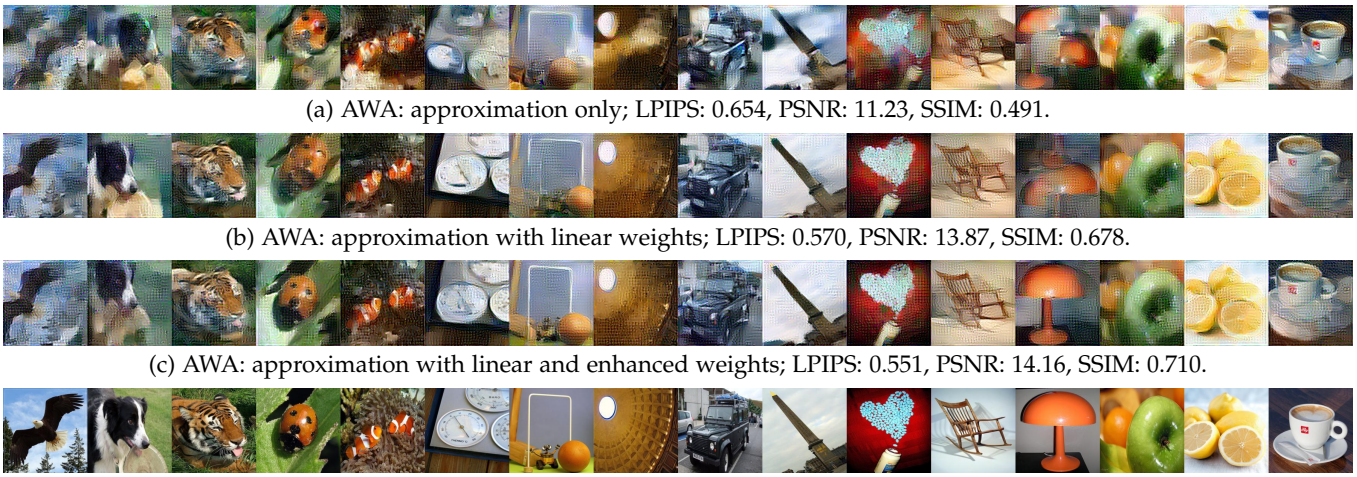
6.3.1 Comparison of Different Attack Methods

In this subsection, we evaluate AWA on the reconstruction of ImageNet images and compare it with other methods under various scenarios. Visual examples are presented in Figure 3, and quantitative metrics are summarized in Table 3.

As shown in Figure 3, AWA reconstructs the main objects with sharper contours and preserves richer texture details. The generalized iDLG approach fails to capture critical features, resulting in blurred or distorted outputs. Although AGIC and GENG reduce some artifacts relative to iDLG, they still lose fine structural information. In contrast, AWA's interpolation-based approximation combined with layer-wise weighted matching produces the most accurate and visually convincing reconstructions.

The numerical results in Table 3 clearly support these visual findings. In each scenario, AWA achieves the highest PSNR and SSIM values and the lowest LPIPS scores, indicating superior pixel-level accuracy and perceptual quality. By comparison, iDLG requires re-running all E local epochs at each attack step, leading to substantially longer runtimes. Similarly, AGIC's strategy of constructing a combined batch of size $N \times E$ imposes high memory demands as N increases. Consequently, for $N \geq 64$, both iDLG and AGIC become impractical.

Overall, AWA consistently achieves superior reconstruction quality across varying numbers of images, epochs,



(a) AWA: approximation only; LPIPS: 0.654, PSNR: 11.23, SSIM: 0.491.

(b) AWA: approximation with linear weights; LPIPS: 0.570, PSNR: 13.87, SSIM: 0.678.

(c) AWA: approximation with linear and enhanced weights; LPIPS: 0.551, PSNR: 14.16, SSIM: 0.710.

(d) Ground truth.

Figure 4: Effect of layer-wise weighting in our AWA method.

TABLE 3: Comparison of four data reconstruction attack methods across various FedAvg scenarios (N images, E epochs, B mini-batches per epoch).

Cases	Metrics	iDLG	AGIC	GENG	AWA (ours)
$N = 8$	PSNR(↑)	8.57	11.13	10.31	**13.49**
$E = 8$	SSIM(↑)	0.019	0.510	0.414	**0.642**
$B = 4$	LPIPS (↓)	0.992	0.659	0.690	**0.573**
$N = 8$	PSNR(↑)	8.57	10.47	10.07	**12.42**
$E = 16$	SSIM(↑)	0.018	0.362	0.345	**0.561**
$B = 4$	LPIPS (↓)	0.993	0.703	0.723	**0.611**
$N = 16$	PSNR(↑)	9.20	10.82	11.08	**12.13**
$E = 8$	SSIM(↑)	0.038	0.362	0.368	**0.569**
$B = 4$	LPIPS (↓)	0.992	0.687	0.700	**0.646**
$N = 16$	PSNR(↑)	9.21	10.45	10.31	**12.33**
$E = 16$	SSIM(↑)	0.039	0.318	0.311	**0.597**
$B = 4$	LPIPS (↓)	0.997	0.707	0.702	**0.642**
$N = 32$	PSNR(↑)	9.26	10.16	9.65	**12.21**
$E = 8$	SSIM(↑)	0.030	0.311	0.237	**0.578**
$B = 4$	LPIPS (↓)	1.084	0.717	0.732	**0.636**
$N = 32$	PSNR(↑)	9.27	9.93	9.53	**12.04**
$E = 16$	SSIM(↑)	0.030	0.243	0.220	**0.548**
$B = 4$	LPIPS (↓)	1.046	0.747	0.744	**0.647**
$N = 64$	PSNR(↑)	–	–	9.14	**11.87**
$E = 32$	SSIM(↑)	–	–	0.023	**0.555**
$B = 4$	LPIPS (↓)	–	–	0.804	**0.680**
$N = 64$	PSNR(↑)	–	–	9.14	**10.95**
$E = 32$	SSIM(↑)	–	–	0.023	**0.451**
$B = 8$	LPIPS (↓)	–	–	0.822	**0.694**
$N = 128$	PSNR(↑)	–	–	9.06	**11.67**
$E = 32$	SSIM(↑)	–	–	0.028	**0.535**
$B = 4$	LPIPS (↓)	–	–	0.804	**0.676**
$N = 128$	PSNR(↑)	–	–	9.06	**10.57**
$E = 32$	SSIM(↑)	–	–	0.028	**0.423**
$B = 8$	LPIPS (↓)	–	–	0.816	**0.702**

and mini-batches, demonstrating robust and scalable performance in diverse FL scenarios.

6.3.2 Effect of Layer-wise Weighting in AWA

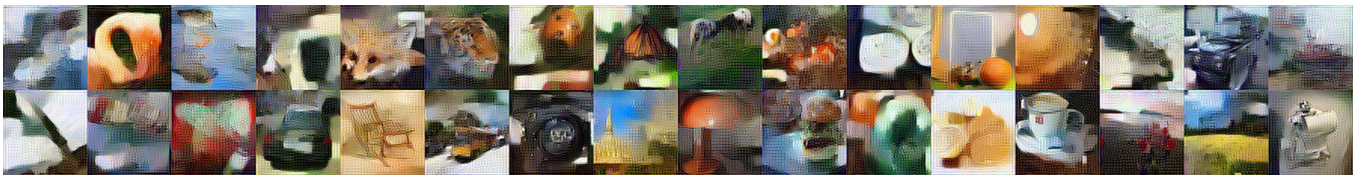
To assess the individual contributions of the approximation step and the weighting scheme in AWA, we compare three settings:

1) **Approximation only.** We apply the epoch-wise interpolation in (15) and set all layer weights to one, ignoring the linearly increasing weights and any error-driven weight enhancement.
2) **Approximation with linear weights.** We keep the interpolation from the previous setting and add linearly increasing weights across layers according to (19). However, we do not include the error-driven enhancement, i.e., $p_{\text{mean}} = p_{\text{var}} = 0$.
3) **Approximation with linear and enhanced weights.** This is the full AWA method. It uses both the layer-wise linearly increasing weights in (19) and the error-driven weight enhancement for high-error layers as in (22).

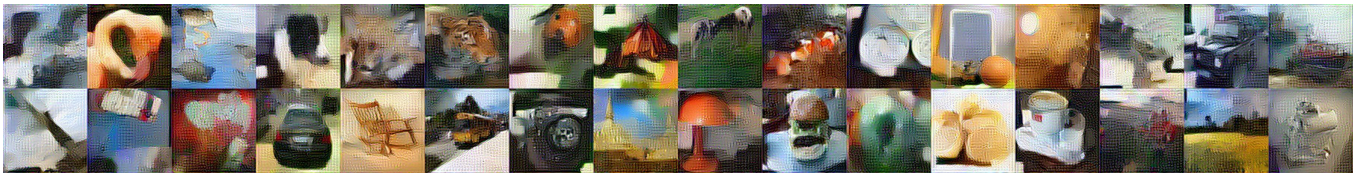
Figure 4 compares the reconstructions for 16 ImageNet images trained over 16 local epochs using a single mini-batch of size 16. Since all three settings use the same interpolation step, this ablation isolates how much of the improvement comes from the weighting design after approximation. The approximation-only baseline recovers only coarse shapes, leading to blurred edges, low PSNR/SSIM scores, and high LPIPS values. Adding linearly increasing weights across network layers significantly sharpens the main contours and improves all three metrics. For instance, the obelisk is reconstructed with noticeably straighter edges. Quantitatively, compared with approximation only, linear weighting improves PSNR by 23.5% and SSIM by 38.1%, while reducing LPIPS by 12.8%. Relative to linear weighting alone, the enhanced weighting further improves PSNR by 2.1% and SSIM by 4.7%, while reducing LPIPS by 3.3%. These results indicate that interpolation is the enabling step, most of the additional gain comes from layer-wise linear weighting, and the error-driven enhancement provides a smaller but consistent refinement in multi-epoch scenarios.

6.3.3 Effect of the Number of Epochs

We further analyze the impact of the number of local training epochs E on reconstruction performance. In a scenario with $N = 32$ images split into $B = 4$ mini-batches, we vary E and present the results in Figure 5. As E increases, the attack becomes more challenging because of stronger model-update aggregation and repeated data shuffling. Nevertheless, AWA maintains high reconstruction quality across all tested values of E, with only a moderate



(a) $E = 32$ epochs; LPIPS: 0.659, PSNR: 11.82, SSIM: 0.528.



(b) $E = 16$ epochs; LPIPS: 0.647, PSNR: 12.04, SSIM: 0.548.



(c) $E = 8$ epochs; LPIPS: 0.636, PSNR: 12.21, SSIM: 0.578.



(d) $E = 4$ epochs; LPIPS: 0.633, PSNR: 12.40, SSIM: 0.600.



(e) Ground truth.

Figure 5: Effect of increasing the number of local epochs on the reconstruction results achieved by our AWA method.

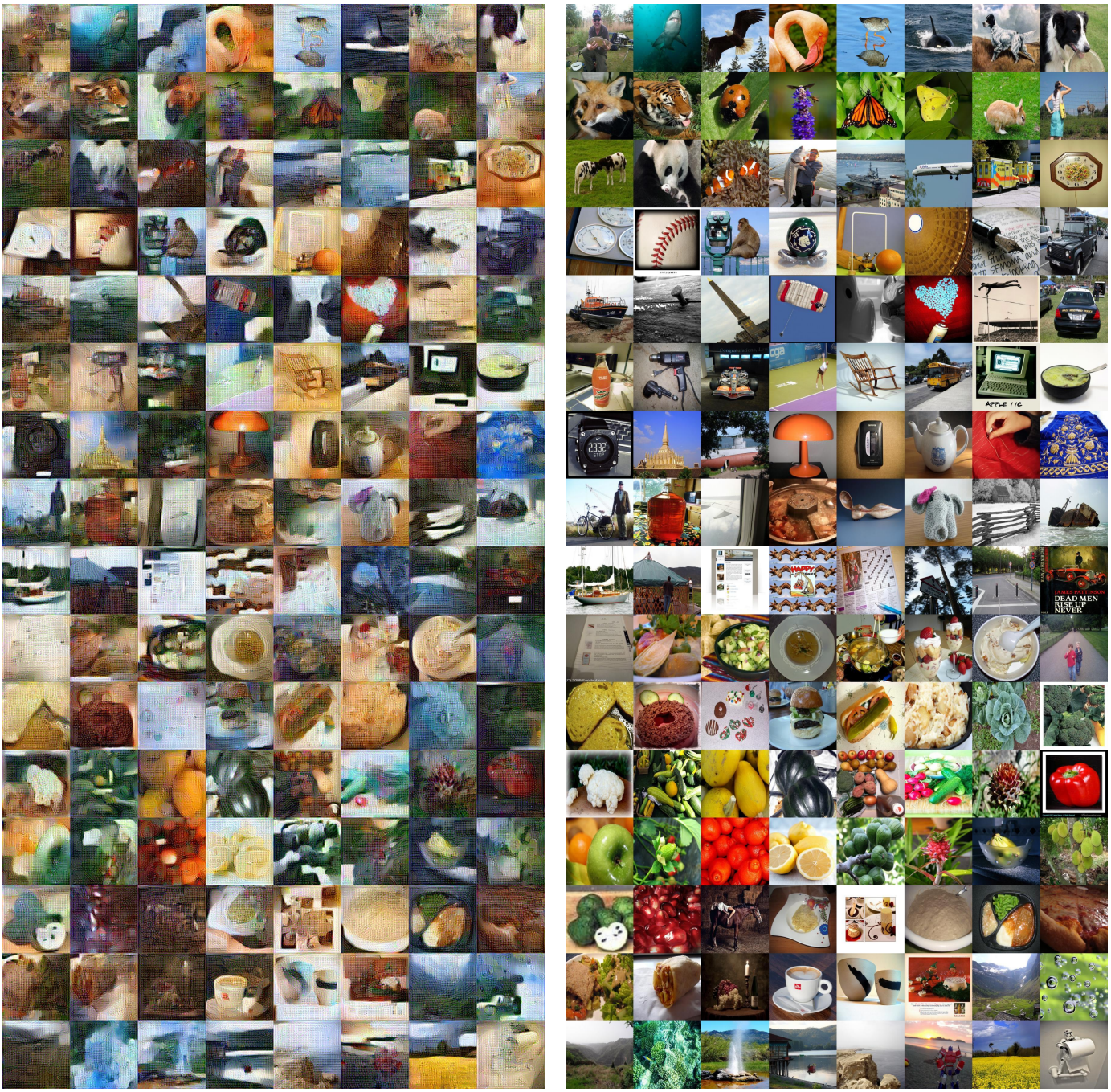
decline in the metrics. This demonstrates the scalability and robustness of our interpolation-based approximation.

6.3.4 *Case Study of a Large-Scale Attack Scenario*

Finally, we evaluate AWA on a large-scale setup with 128 ImageNet images trained for 64 local epochs and split into four mini-batches per epoch, each with a mini-batch size of 32. To handle the added complexity, we increase the number of attack iterations from 10,000 to 50,000. As shown in Figure 6, AWA can still faithfully reconstruct the main visual structures and recover object categories, demonstrating its robustness in FL scenarios with large local datasets and extensive local training. Overall, these results further validate the effectiveness of AWA across diverse and realistic FedAvg scenarios.

7 Conclusions

The privacy benefits of FL are threatened by recently developed data reconstruction attacks. In this paper, we formulated the attack as an inverse problem, iteratively reconstructing the client's private training data by solving an optimization problem. To attack the widely used FedAvg algorithm, we proposed an interpolation-based approximation method, in which the intermediate model updates corresponding to each local epoch are effectively estimated via parameter interpolation. Furthermore, we introduced a layer-wise weighted loss function to substantially enhance reconstruction quality. By systematically assigning appropriate weights to model updates across different neural network layers via Bayesian optimization, our approach recovers training data with higher fidelity than existing state-of-the-art methods. Notably, our weighting strategy is architecture-aware, ensuring broad applicability rather than being restricted to a specific network design. Extensive numerical results demonstrated that our approximate and weighted data reconstruction attack is highly effective in realistic FedAvg scenarios, exposing critical privacy vulnerabilities in current FL systems. The ability to faithfully reconstruct raw data from leaked model updates underscores an immediate need for robust defense mechanisms. Future research will focus on extending these empirical evaluations to diverse architectures, developing rigorous countermeasures, and strengthening the security guarantees of FL frameworks to mitigate the severe risks associated with such privacy breaches.



(a) AWA (ours); LPIPS: 0.680, PSNR: 11.799, SSIM: 0.546.

(b) Ground truth.

Figure 6: Results of our AWA method on 128 ImageNet images trained for 64 local epochs with a mini-batch size of 32.

ACKNOWLEDGMENTS

The authors' names are listed in alphabetical order by family name to signify equal contributions. Z. Wang was supported by the European Union's Horizon Europe MSCA project ModConFlex (HORIZON-MSCA-2021-DN-01 project 101073558). Y. Song was supported by the NTU Start-Up Grant. E. Zuazua was partially supported by the European Research Council (ERC) under the European Union's Horizon 2030 research and innovation programme (grant agreement NO: 101096251-CoDeFeL); by the Alexander von Humboldt Professorship program; the European Union's Horizon Europe MSCA project ModConFlex (HORIZON-MSCA-2021-DN-01 project 101073558); the Transregio 154 Project "Mathematical Modelling, Simulation and Optimization Using the Example of Gas Networks" of the DFG; the AFOSR 24IOE027 project; the SURE-AI Norwegian Centre for Sustainable, Risk-Averse, and Ethical AI grant 357482, Research Council of Norway; by the Grant PID2023-146872OB-I00-DyCMaMod of MICIU (Spain) and by the COST Actions CA24122 - Multiscale Stochastics, Patterns, and Analysis of Combinatorial Environments and CA24136 - Interactions between Control Theory and Machine Learning.

REFERENCES

[1] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, "Federated Learning: Challenges, Methods, and Future Directions," *IEEE Signal Processing Magazine*, vol. 37, no. 3, pp. 50–60, 2020, ISSN: 1558-0792.

[2] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-Efficient Learning of Deep Networks from Decentralized Data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, PMLR, 2017, pp. 1273–1282.

[3] T. S. Brisimi, R. Chen, T. Mela, A. Olshevsky, I. C. Paschalidis, and W. Shi, "Federated learning of predictive models from federated Electronic Health Records," *International Journal of Medical Informatics*, vol. 112, pp. 59–67, 2018, ISSN: 1386-5056.

[4] J. Xu, B. S. Glicksberg, C. Su, P. Walker, J. Bian, and F. Wang, "Federated Learning for Healthcare Informatics," *Journal of Healthcare Informatics Research*, vol. 5, no. 1, pp. 1–19, 2021, ISSN: 2509-498X.

[5] T. Zeng, O. Semiari, M. Chen, W. Saad, and M. Bennis, "Federated Learning on the Road Autonomous Controller Design for Connected and Autonomous Vehicles," *IEEE Transactions on Wireless Communications*, vol. 21, no. 12, pp. 10 407–10 423, 2022, ISSN: 1558-2248.

[6] J. Geng et al., "Improved Gradient Inversion Attacks and Defenses in Federated Learning," *IEEE Transactions on Big Data*, pp. 1–13, 2023, ISSN: 2332-7790.

[7] J. Xu, C. Hong, J. Huang, L. Y. Chen, and J. Decouchant, "AGIC: Approximate Gradient Inversion Attack on Federated Learning," in *2022 41st International Symposium on Reliable Distributed Systems (SRDS)*, 2022, pp. 12–22.

[8] H. Yin, A. Mallya, A. Vahdat, J. M. Alvarez, J. Kautz, and P. Molchanov, "See through gradients: Image batch recovery via gradinversion," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 16 337–16 346.

[9] L. Zhu, Z. Liu, and S. Han, "Deep Leakage from Gradients," in *Advances in Neural Information Processing Systems*, vol. 32, Curran Associates, Inc., 2019.

[10] P. I. Frazier, "A Tutorial on Bayesian Optimization," *arXiv preprint arXiv:1807.02811*, 2018.

[11] B. Zhao, K. R. Mopuri, and H. Bilen, "iDLG: Improved Deep Leakage from Gradients," *arXiv preprint arXiv:2001.02610*, 2020.

[12] J. Geiping, H. Bauermeister, H. Dröge, and M. Moeller, "Inverting gradients-how easy is it to break privacy in federated learning?" *Advances in neural information processing systems*, vol. 33, pp. 16 937–16 947, 2020.

[13] Y. Wang et al., "SAPAG: A Self-Adaptive Privacy Attack From Gradients," *arXiv preprint arXiv:2009.06228*, 2020.

[14] D. C. Liu and J. Nocedal, "On the limited memory BFGS method for large scale optimization," *Mathematical Programming*, vol. 45, no. 1, pp. 503–528, 1989, ISSN: 1436-4646.

[15] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.

[16] W. Wei et al., "A framework for evaluating client privacy leakages in federated learning," in *European Symposium on Research in Computer Security*, Springer, 2020, pp. 545–566.

[17] G. B. Huang, M. Mattar, T. Berg, and E. Learned-Miller, "Labeled Faces in the Wild: A Database forStudying Face Recognition in Unconstrained Environments," in *Workshop on Faces in 'Real-Life' Images: Detection, Alignment, and Recognition*, 2008.

[18] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," 2009.

[19] J. Jeon, J. Kim, K. Lee, S. Oh, and J. Ok, "Gradient Inversion with Generative Image Prior," in *Advances in Neural Information Processing Systems*, vol. 34, Curran Associates, Inc., 2021, pp. 29 898–29 908.

[20] O. Calin, *Deep Learning Architectures: A Mathematical Approach* (Springer Series in the Data Sciences). Cham: Springer International Publishing, 2020, ISBN: 978-3-030-36720-6 978-3-030-36721-3.

[21] S. Chen and Q. Zhao, "Shallowing Deep Networks: Layer-Wise Pruning Based on Feature Representations," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 41, no. 12, pp. 3048–3056, 2019, ISSN: 1939-3539.

[22] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.

[23] C. E. Rasmussen and C. K. I. Williams, *Gaussian Processes for Machine Learning* (Adaptive Computation and Machine Learning). Cambridge, Mass: MIT Press, 2006, ISBN: 978-0-262-18253-9.

[24] J. Močkus, "On Bayesian methods for seeking the extremum," in *Optimization Techniques IFIP Technical Conference: Novosibirsk, July 1–7, 1974*, Springer, 1975, pp. 400–404, ISBN: 3-662-37713-6.

[25] D. R. Jones, M. Schonlau, and W. J. Welch, "Efficient Global Optimization of Expensive Black-Box Functions," *Journal of Global Optimization*, vol. 13, no. 4, pp. 455–492, 1998, ISSN: 1573-2916.

[26] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *2009 IEEE conference on computer vision and pattern recognition*, Ieee, 2009, pp. 248–255.

[27] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 586–595.

[28] U. Sara, M. Akter, and M. S. Uddin, "Image Quality Assessment through FSIM, SSIM, MSE and PSNR—A Comparative Study," *Journal of Computer and Communications*, vol. 7, no. 3, pp. 8–18, 2019.

[29] Z. Wang, A. Bovik, H. Sheikh, and E. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004, ISSN: 1941-0042.